Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине: Контейнеризация и оркестрация приложений

Тема работы: Персистентность и изоляция: добавляем БД и настраиваем сети

Обучающийся: Мануковская Дарья Михайловна

Номер ИСУ: 466625

Факультет: Факультет прикладной информатики

Группа: К3321

Направление подготовки: 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа: Программирование в инфокоммуникационных системах

Обучающийся _____ Мануковская Д. М.
дата, подпись

Руководитель _____ Аминов Н. С.
дата, подпись

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Ход выполнения **4**

1.1 Запуск контейнера с именованным volume 4

1.2 Тест персистентности: пересоздание контейнера 5

1.3 Bind mount для конфигурации 6

1.4 Пользовательские сети для изоляции 7

1.5 Финальный артефакт 8

1.6 Ошибки и анализ результатов 9

ЗАКЛЮЧЕНИЕ **10**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **11**

ПРИЛОЖЕНИЕ А Дополнительные команды и конфигурации **12**

ВВЕДЕНИЕ

Цель: обрать свой образ приложения, уменьшить его размер и добавить проверку здоровья. Для этого нужно собрать образ по Dockerfile, сравнить размер обычной и уменьшенной сборки и настроить healthcheck, чтобы контейнер сообщал о своем состоянии.

Таблица 1 — Персональные параметры

Параметр	Значение
Номер ИСУ	466625
Персональный порт	5455, 8025
Персональный префикс	manuk
Секретный ключ	secret25manuk
Имя баз данных	manuk-postgres

Результат хранится в ветке `lab-3` своего форка: https://github.com/wecanallreborn/KIOP/tree/lab-3.

1 Ход выполнения

1.1 Запуск контейнера с именованным volume

Создаем том и проверяем его созданием следующими командами. (рисунок 1)

```
docker volume create manuk-pgdata
docker volume ls | grep manuk-pgdata
docker volume inspect manuk-pgdata
```

```
dariamanuk@MacBook-Air-Daria-2 course-containerization-orchestration-application % docker volume create manuk-pgdata
manuk-pgdata
dariamanuk@MacBook-Air-Daria-2 course-containerization-orchestration-application % docker volume ls | grep manuk-pgdata
local     manuk-pgdata
dariamanuk@MacBook-Air-Daria-2 course-containerization-orchestration-application % docker volume inspect manuk-pgdata
[
    {
        "CreatedAt": "2026-11-06T10:40:59Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/manuk-pgdata/_data",
        "Name": "manuk-pgdata",
        "Options": null,
        "Scope": "local"
    }
```

Рисунок 1 — Терминальный вывод

Запускаем PostrgesSql и проверяем статус (рисунок 2).

```
docker run -d \
--name manuk-postgres \
-e POSTGRES_USER=user_25 \
-e POSTGRES_PASSWORD=pgpass_25 \
-e POSTGRES_DB=assistant_25 \
-v manuk-pgdata:/var/lib/postgresql/data \
-p 5455:5432 \
postgres:15

docker ps --filter "name=manuk-postgres" --format "table
↪  {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
NAMES            STATUS          PORTS
manuk-postgres   Up 48 seconds   0.0.0.0:5455->5432/tcp, [::]:5455->5432/tcp
```

Рисунок 2 — Терминальный вывод

В ЛР2 уже собирали образ v3, поэтому пересобирать его не нужно. Запускаем ассистента и подключаем к базе данных.

```
docker run -d \
  --name manuk-assistant \
  -p 8025:8025 \
  -e PORT=8025 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  -e DATABASE_URL="postgresql://user_25:pgpass_25@host.docker.intern
   ↪  al:5455/assistant_25" \
  manuk-assistant:v3
```

Проверим работоспособность. Результаты работы команд представлены на рисунке 3.

```
curl http://localhost:8025/
curl http://localhost:8025/health
curl -X POST http://localhost:8025/notes -H "Content-Type:
 ↪  application/json" -d '{"text": "ЛР3: данные в БД"}'
curl http://localhost:8025/notes
```



Рисунок 3 — Терминальный вывод

1.2 Тест персистентности: пересоздание контейнера

Удаляем контейнер с ассистентом

```
docker stop manuk-assistant
docker rm manuk-assistant
```

Пересоздаем контейнер ассистента и проверяем сохранение данных (рисунок 4)

```
docker run -d \
--name manuk-assistant \
-p 8025:8025 \
-e APP_OWNER=manuk \
-e APP_SECRET=secret25manuk \
```

```
-e PORT=8025 \
-e DATABASE_URL="postgresql://user_25:pgpass_25@host.docker.internal
 ↪  :5455/assistant_25" \
manuk-assistant:v3
curl http://localhost:8025/notes
```

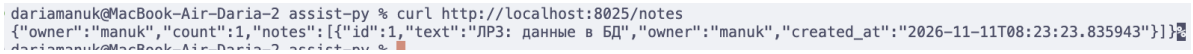


Рисунок 4 — Терминальный вывод

Артефакт персистентности представлен на рисунке 5

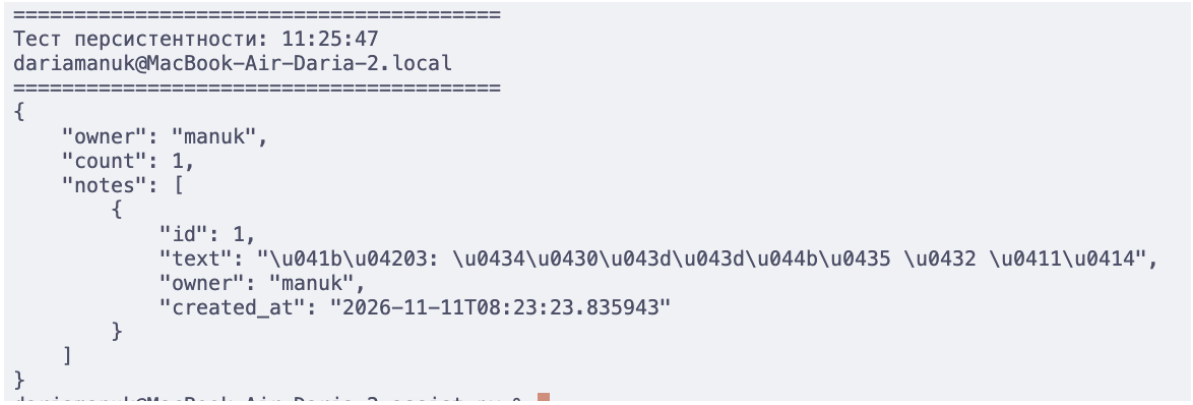


Рисунок 5 — Терминальный вывод

1.3 Bind mount для конфигурации

Изменили конфигурационный файл приложения. Собираем контейнер с bind mount.

```
docker run -d \
  --name manuk-assistant-config \
  -p 8025:8025 \
  -e PORT=8025 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  --add-host=host.docker.internal:host-gateway \
  -e DATABASE_URL="postgresql://user_25:pgpass_25@host.docker.intern
   ↪  al:5455/assistant_25" \
  -v $(pwd)/config:/app/config:ro \
  manuk-assistant:v3
```

Проверили доступ к конфигурации 6.

```
docker exec -it manuk-assistant-config sh
cat /app/config/app.conf
```

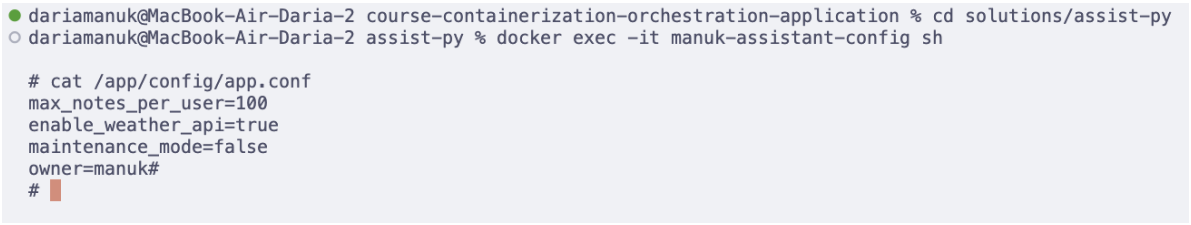


Рисунок 6 — Терминальный вывод

Bind mount позволяет менять конфигурацию без пересборки образа.

1.4 Пользовательские сети для изоляции

```
docker network create manuk-backend
```

Пересоздаем контейнеры в пользовательской сети.

```
docker stop manuk-postgres manuk-assistant-config 2>/dev/null
docker rm manuk-postgres manuk-assistant-config 2>/dev/null

docker run -d \
  --name manuk-postgres \
  --network manuk-backend \
  -e POSTGRES_USER=user_25 \
  -e POSTGRES_PASSWORD=pgpass_25 \
  -e POSTGRES_DB=assistant_25 \
  -v manuk-pgdata:/var/lib/postgresql/data \
  postgres:15

  docker run -d \
  --name manuk-assistant \
  --network manuk-backend \
  -p 8025:8025 \
  -e PORT=8025 \
  -e APP_OWNER=manuk \
  -e APP_SECRET=secret25manuk \
  -e DATABASE_URL="postgresql://user_25:pgpass_25@manuk-postgres:543
  ↪  2/assistant_25" \
```

```
  -v $(pwd)/config:/app/config:ro \
  manuk-assistant:v3
```

Проверяем связность контейнеров. (рисунок 7)

```
curl http://localhost:8025/health
curl -X POST http://localhost:8025/notes \
-H "Content-Type: application/json" \
-d '{"text": "ЛР3: сеть работает!"}'
curl http://localhost:8025/notes
```



Рисунок 7 — Терминальный вывод

Демонстрация сетевой изоляции. (рисунок 8)

```
docker run -it --rm alpine:3.18 ping -c 2 manuk-postgres
docker run -it --rm --network manuk-backend alpine:3.18 ping -c 2
↪  manuk-postgres
```

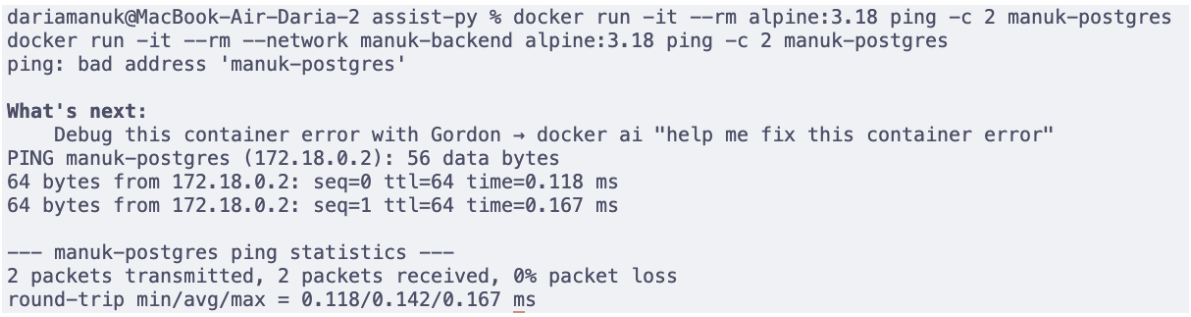


Рисунок 8 — Терминальный вывод

1.5 Финальный артефакт

Вставьте собственный скриншот

pic/f.png

Эта рамка не является результатом выполнения работы.

Рисунок 9 — Терминальный вывод

1.6 Ошибки и анализ результатов

Тк в сборке образа брали переменную PORT для подстановки со значением 5455. Приложение слушает порт 5455 внутри контейнера, а наружу проброшен 8025. Для исправления при сборке контейнера переопределим значение переменной -e PORT=8025.

```
dariamanuk@MacBook-Air-Daria-2 course-containerization-orchestration-application % curl http://localhost:8025/
curl http://localhost:8025/health
curl -X POST http://localhost:8025/notes -H "Content-Type: application/json" -d '{"text": "ЛР3: данные в БД"}'
curl http://localhost:8025/notes
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
curl: (56) Recv failure: Connection reset by peer
```

Рисунок 10 — Терминальный вывод

ЗАКЛЮЧЕНИЕ

Свой образ приложения собран, его размер уменьшен, проверка здоровья добавлена и проверена. Поставленная цель достигнута: приложение запускается из собственного образа, образ стал компактнее, а состояние контейнера контролируется через healthcheck.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Docker Documentation: документация Docker. URL: https://docs.docker.com/ (дата обращения: укажите свою дату).

ПРИЛОЖЕНИЕ А
Дополнительные команды и конфигурации

Включите объёмные листинги, на которые ссылаетесь в основной части. Короткий пример ниже показывает оформление, но не заменяет ваш код.

```
# Свой безопасный пример конфигурации:
APP_SECRET=REDACTED
```